

FullCycle

Autenticação e autorização: Tokens JWT, ACL e RBAC



Conteúdo do Curso

Introdução à
Autenticação e
Autorização

1

2

Anatomia do JWT

JWT na Prática

3

Padrões de autorização

4

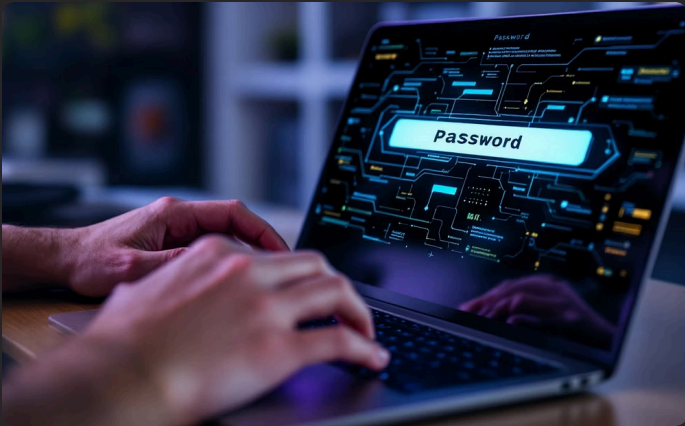
Segurança e Melhores
Práticas

5

A Chave para a Segurança: Autenticação

Imagine um mundo onde qualquer um pudesse se passar por você e acessar suas informações confidenciais. É aí que entra a autenticação - o processo de verificar a sua identidade de forma segura.

Existem três principais formas de autenticação:



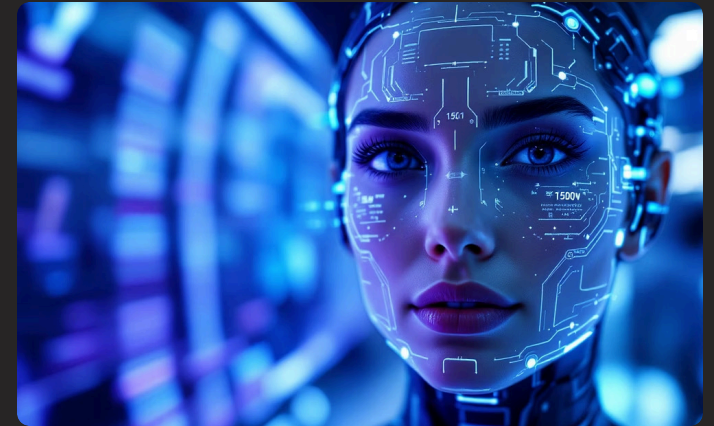
Conhecimento

Algo que você sabe, como uma senha ou PIN.



Posse

Algo que você tem, como um token ou chaveiro.



Inerência

Algo que você é, como sua impressão digital ou reconhecimento facial.

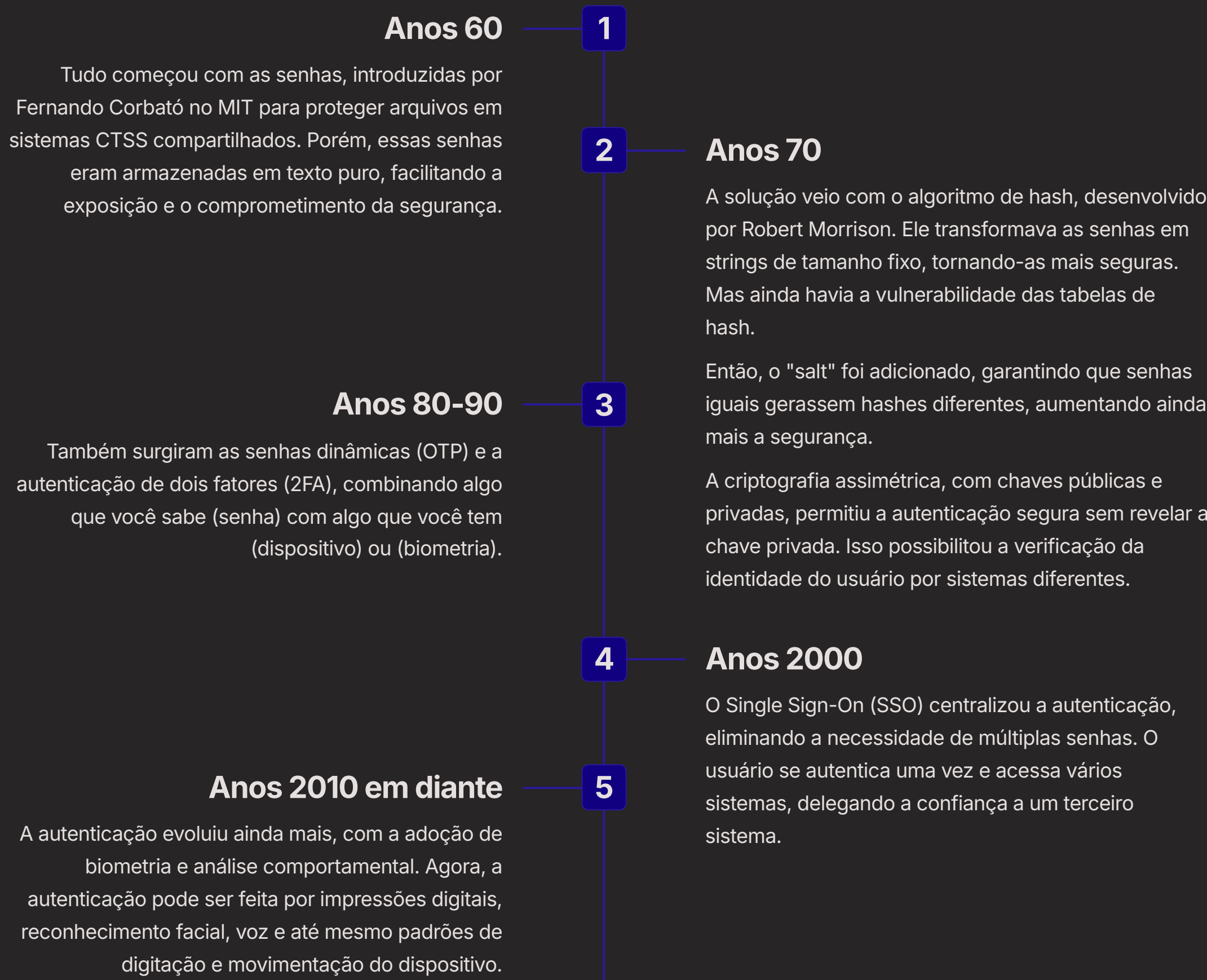
Mas não se engane - a autenticação sozinha não é suficiente. Para realmente garantir a segurança, é preciso implementar a autenticação de múltiplos fatores (MFA). Isso significa combinar duas ou mais dessas formas de autenticação, tornando muito mais difícil para alguém se passar por você.

Então, a próxima vez que você fizer login em uma conta, lembre-se: a autenticação é a chave para proteger suas informações e sua identidade. Quanto mais camadas de segurança, melhor!

História da autenticação

A Evolução da Autenticação: De Senhas a Biometria

Ao longo das décadas, a autenticação tem sido um campo em constante evolução, acompanhando as crescentes demandas por segurança da informação. Vamos explorar essa fascinante jornada e entender como chegamos até aqui.



A autenticação é um campo vasto e em constante evolução, mas o objetivo sempre foi o mesmo: garantir que a pessoa é quem diz ser. E essa jornada é fascinante de acompanhar!

O que é Autorização?

Além da Autenticação: O Poder da Autorização

A autenticação confirma sua identidade, mas a autorização define o que você pode fazer. É como a chave do seu quarto de hotel: você precisa ser autenticado (identificado) para receber a chave (autorização) e acessar seu quarto.



A Chave do Hotel

A autenticação é como se apresentar na recepção. A autorização é receber a chave para acessar seu quarto.



Níveis de Acesso

Clientes e funcionários são autenticados, mas têm permissões diferentes. Funcionários têm acesso a mais áreas do sistema do que clientes.



Mecanismos de Autorização

Existem métodos como ACLs, RBAC e ABAC para controlar o acesso. A escolha depende das necessidades do sistema.

Autorização é essencial para sistemas seguros. Vamos explorar mais!

Diferenças entre autenticação e autorização



Quem é Você? (Autenticação)

Verificar a identidade do usuário. Como um guarda-costas checando seu RG.



O Que Você Pode Fazer? (Autorização)

Determinar o que um usuário autenticado pode fazer. Como uma catraca que libera acesso a áreas específicas.



Segurança em Camadas

Primeiro, quem está acessando? Depois, o que essa pessoa pode fazer. Dois passos essenciais.

Entendido a diferença? Explore os mecanismos de autorização nas próximas aulas!

Autenticação na WEB

A web é um ambiente aberto, onde qualquer um pode acessar nossos sistemas. A autenticação é o primeiro passo para proteger nossos usuários e recursos.

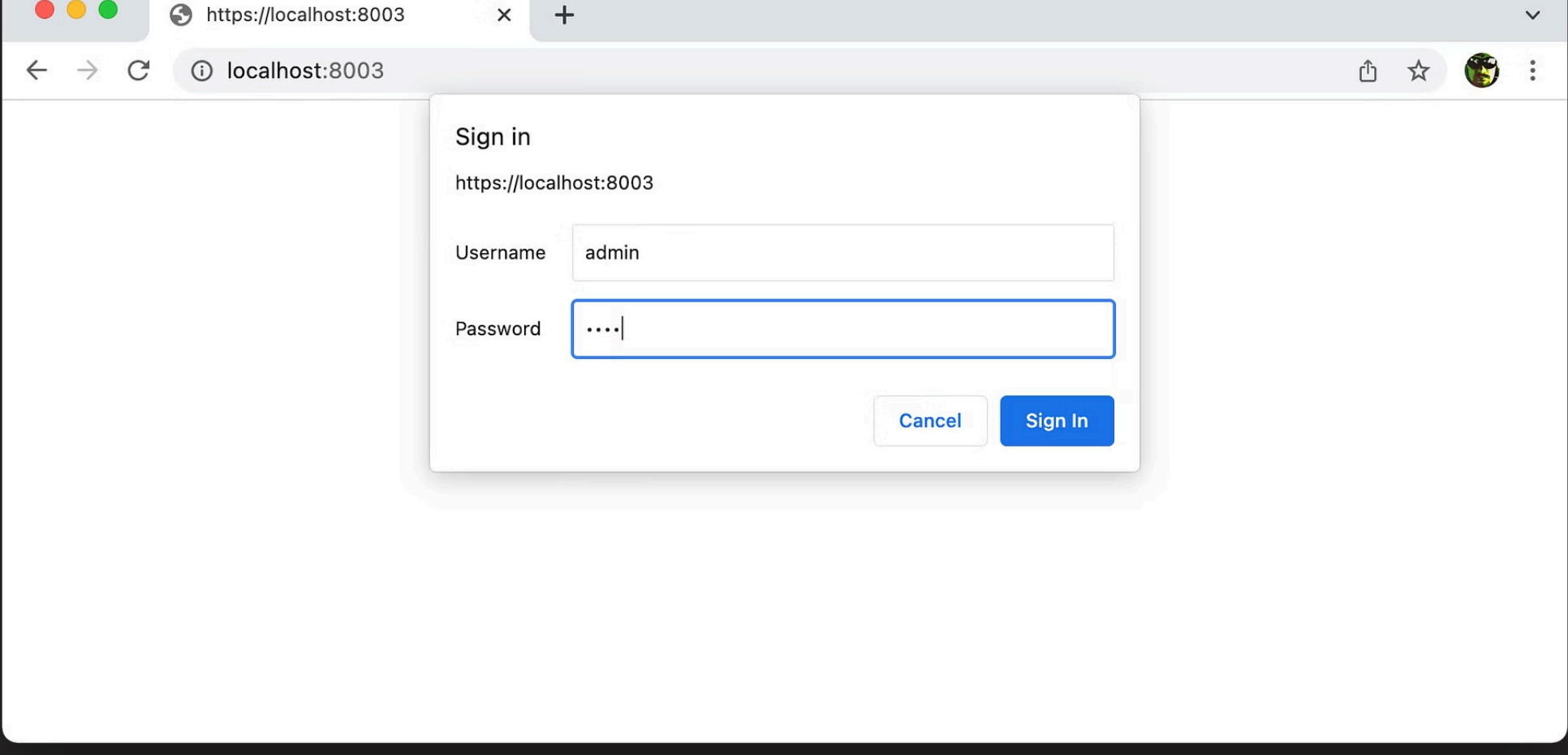
Autenticação se refere às informações que o usuário precisa fornecer para se identificar, como email, senha, biometria ou certificados digitais. Mas o desafio é manter o usuário autenticado entre as requisições, já que o HTTP é um protocolo stateless.

Métodos como Basic Auth, Digest Auth, Sessões e Tokens ajudam a resolver esse problema, cada um com suas vantagens e desvantagens.

Métodos de Autenticação

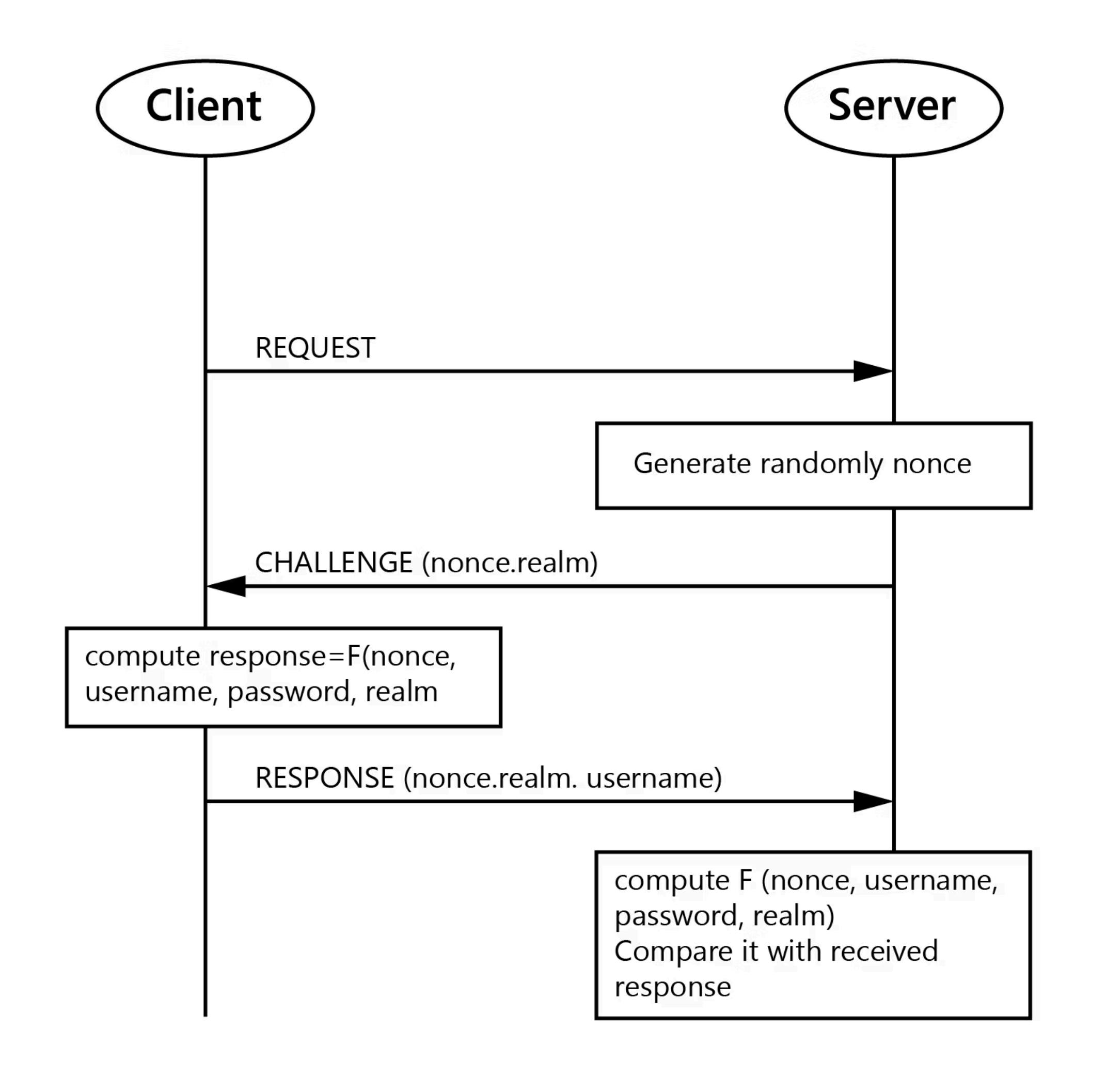
Temos várias formas de manter o usuário autenticado, como:

Basic Authentication



Basic authentication: é um método de autenticação simples, em que o usuário informa seu email e senha. Ao clicar no botão de login, o navegador codifica o email e senha em base64 e envia uma requisição com header Authorization: Basic base64(email:senha). Após o servidor validar as credenciais, as próximas requisições continuam enviando este header Authorization: Basic base64(email:senha) da mesma forma, é isto mesmo, o email e senha são enviados em todas as requisições. A senha está totalmente exposta e sempre o servidor precisa validar as credenciais. Para se ter um pouco mais de segura, é necessário usar HTTPS para criptografar a comunicação. Normalmente todos os browsers tem suporte a ela, até mesmo a tela de login é exibida pelo browser. É usado ainda por sistema mais antigos e interfaces WEB de roteadores mais antigos também.

Digest Authentication



Digest authentication: é um método de autenticação parecido com o Basic, mas a informação enviada no header Authorization é criptografada. Quando o usuário acessa um endereço, o servidor responde assim:

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Digest realm="meu-servidor",
    nonce="abc123",
    qop="auth",
    opaque="xyz456"
```

Então, o browser ou cliente em posse da informação, usa as informações para criar um hash e enviar no header Authorization. Um exemplo:

```
Authorization: Digest username="luiz", realm="meu-servidor", nonce="abc123", uri="/", response="hash", opaque="xyz456"
```

O servidor então valida o hash e se estiver correto, o usuário é autenticado, então ele tem que enviar estas informações sempre em todas as requisições, como no Basic Auth. O Digest é mais seguro que o Basic, pois não expõe a senha, mas ainda é vulnerável a ataques como Man-in-the-middle, em que o atacante pode interceptar a requisição/resposta e enganar o cliente a usar basic auth ou um padrão de hash fraco.

Sessão e Cookie

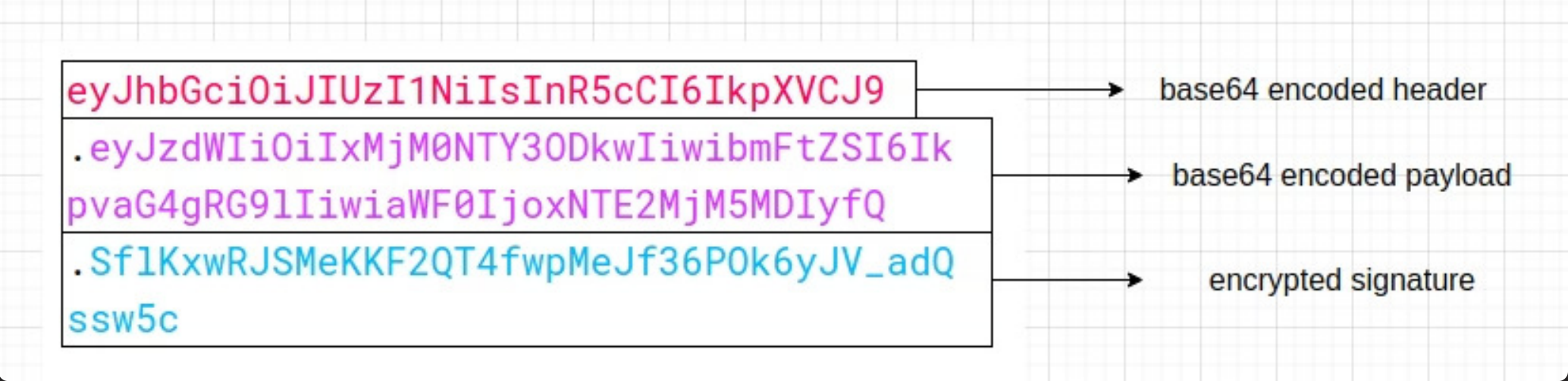


A sessão e o cookie são métodos de autenticação amplamente utilizados, mas não são perfeitos. Quando o usuário efetua o login, o servidor gera um cookie único que é armazenado no navegador. A cada acesso, esse cookie é enviado de volta ao servidor, que o valida para autenticar o usuário. Embora seja uma solução simples e eficaz, esse método tem algumas desvantagens:

- 1 Stateful**
O servidor precisa armazenar o estado da autenticação do usuário, o que pode ser um problema para a escalabilidade da aplicação.
- 2 Vulnerável a ataques**
O cookie pode ser alvo de ataques como CSRF, onde um link malicioso pode ser usado para fazer requisições em nome do usuário.
- 3 Limitado a navegadores**
Por ser um conceito específico do navegador, esse método de autenticação fica restrito a aplicações web tradicionais.

Apesar dessas limitações, a sessão e o cookie continuam sendo uma das formas mais utilizadas de autenticação na web.

Tokens



- 1 Versatilidade dos Tokens**
Tokens podem ser usados por qualquer tipo de cliente - navegadores, desktops, smartphones, e muito mais!
- 2 Autenticação Simplificada**
O servidor gera um token exclusivo após receber as credenciais do usuário. Esse token é armazenado e enviado em todas as requisições, tornando a autenticação rápida e segura.
- 3 Diversidade de Tokens**
Tokens podem ser armazenados de várias formas (localStorage, cookies, etc.) e existem diferentes tipos, como os opacos e os JWT.

História do JWT

1 — Outros métodos de autenticação

Vários métodos de autenticação existiam antes do JWT, como Basic Auth, Digest Auth, Sessão e Cookie, etc. No entanto, todos esses métodos tinham limitações.

2 — Autenticação por Tokens e Token opacos

Antes do JWT, eram utilizados tokens opacos, sem informações, gerados pelo servidor. Cada um gerava seu próprio token e o armazenava no servidor.

3 — Nasce o padrão JWT

O padrão JWT foi publicado em 28 de dezembro de 2010, usando o formato JSON. Oficializado pela IETF em 2015, o JWT faz parte da família do JOSE.

4 — Adoção do JWT

O JWT começou a ser amplamente adotado pelas empresas e fez parte de padrões de mercado como OAuth, OpenID Connect, etc.

5 — Benefícios do JWT

O JWT permitiu comunicação stateless entre sistemas, resultando em eficiência, segurança e escalabilidade.

The JWNT logo features a stylized starburst icon on the left, composed of eight colored segments (cyan, magenta, red, and blue) radiating from a central white point. To the right of the icon, the letters 'JWNT' are displayed in a large, bold, black, sans-serif font.

```

left: 10px;
-right: 10px;
g-left: 10px;
g-right: 10px;
weight: bold;
low: auto;

login id="login_form" >
it color='red'><b>Authentication Failed</b></b></b>
"dError1" class="dError1">Please contact the administrator for help

<saml-auth-status>-1</saml-auth-status>

window.top.location='/php/login.php';return false;

```

[illegible]

Assinatura gerada a partir do cabeçalho, das reivindicações e de uma chave secreta. Garante a integridade do token.

JWT Header

O cabeçalho é a primeira parte do token e contém o tipo do token e o algoritmo de criptografia usado para assinar o token. O cabeçalho é codificado em base64-url, como falamos na aula anterior e é a menor parte do token.

O header é composto de `alg` e `typ`. Utilizando a especificação da família JOSE, é aqui que entra a especificação JWA que define os padrões de algoritmos de criptografia que serão usados no token, o JWE (JSON Web Encryption) se necessitamos criptografar o conteúdo do token e o JWS (JSON Web Signature) se necessitamos assinar o token, claro que sempre queremos um token assinado para ter mais segurança, mas podemos ter um token sem assinatura, isto é usado em ambientes que não precisam de segurança, é mais raro, mas pode acontecer.



alg: Algoritmo de Criptografia

Define o algoritmo usado para assinar o token (e.g., HS256, RS256). Informa como verificar a assinatura. HS256 (HMAC + SHA256) é mais simples, ideal para comunicação direta entre back-end e front-end. RS256 (RSA) é assimétrico, permite validação por várias aplicações usando a chave pública.



typ: Tipo de Token

Indica o tipo de token, geralmente "JWT". Informa que o token segue os padrões JWT. Outros tipos, como "JWE", indicam criptografia no payload.

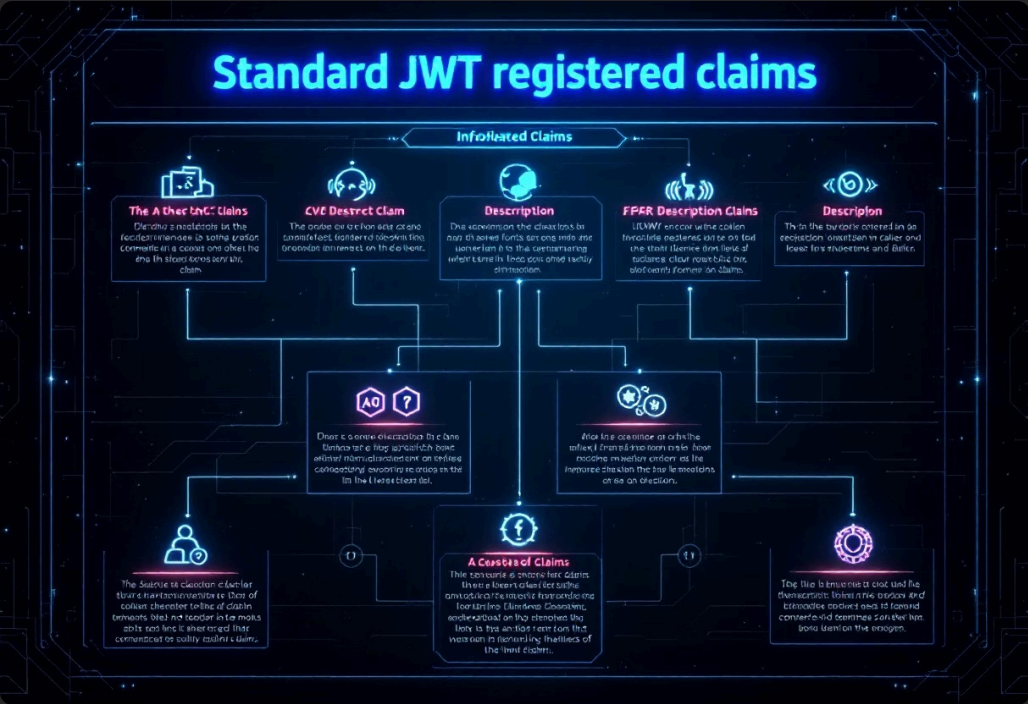
JWT Payload

O payload é a segunda parte do token. O payload contém informações diversas, como nome do usuário, email, permissões etc., além das "registered claims", informações padrão (criação, expiração etc.). Estas informações são definidas pela RFC 7519 (especificação JWT).

As informações dentro do payload são chamadas de reivindicações ("claims"). "Claims" representa as informações que o token está reivindicando (o token afirma que o usuário é João, tem tal email, expira em tal horário etc.).

As reivindicações são divididas em 3 tipos:

Registered claims:



Reivindicações padrão (iss, sub, aud, exp, nbf, iat, jti). Definidas pela RFC JWT e comuns a todos os tokens. Seu uso depende do nível de segurança e especificidade desejado. Ex: se não importa o tempo de expiração, não use `exp`. Os nomes são definidos pela RFC para evitar conflitos; novos nomes devem ser registrados na IANA.

```
{
  "iss": "meu-servidor",
  "sub": "1234567890",
  "exp": 1516239022
  ...
}
```

Private claims:



Criadas para um sistema específico (permissões, roles etc.). Não devem conflitar com as reivindicações registradas nem conter informações sensíveis. Seu uso é específico de um sistema.

```
{
  "iss": "meu-servidor",
  "sub": "1234567890",
  "exp": 1516239022,
  "name": "John Doe",
  "email": "john@email",
  ...
  "role": "admin",
  "permissions": [
    {
      "resource": "users",
      "actions": ["read", "write"]
    },
    {
      "resource": "products",
      "actions": ["read"]
    }
  ]
}
```



Public claims:



Definidas pelo usuário (nome, email etc.). Podem ser quaisquer informações, desde que não conflitem com as reivindicações registradas e não sejam sensíveis (o payload é acessível a qualquer um).

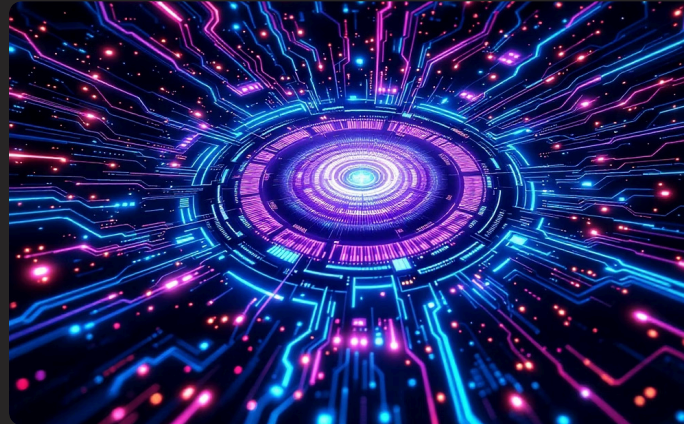
```
{
  "iss": "meu-servidor",
  "sub": "1234567890",
  "exp": 1516239022,
  ...
  "name": "John Doe",
  "email": "john@email"
  ...
}
```

JWT Signature



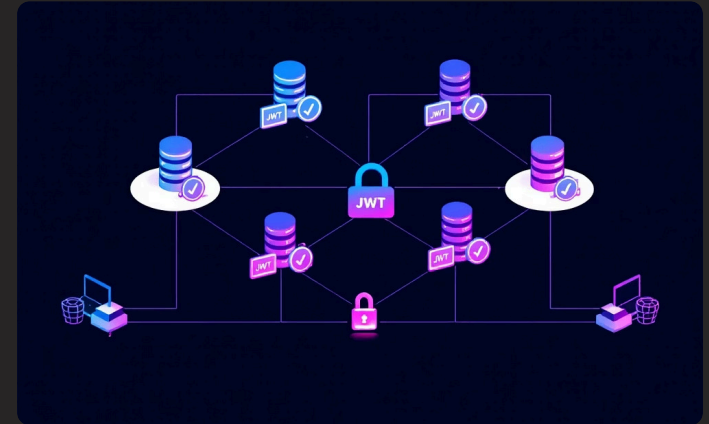
Assinatura JWT: Garantia de Autenticidade

A assinatura garante que o token foi emitido pela fonte correta e não foi modificado.



Algoritmos de Criptografia

Usamos JWS e JWA para definir os algoritmos (HS256, RS256, etc.), garantindo a integridade.



Escolha do Algoritmo

A escolha depende do contexto (sistemas monolíticos vs. distribuídos). RS256 é ideal para sistemas distribuídos.

Modelos de Autorização

- **RBAC:** Baseado em roles, simplifica a gestão de permissões.
- **ACL:** Listas de controle de acesso detalhadas e flexíveis.
- **ABAC:** Autorização baseada em atributos, alta customização.
- **DAC:** Controle discricionário, usuários definem acesso.
- **MAC:** Controle mandatório, sistema define o acesso.

RBAC: Role-based Access Control

Usuários

Entidades que interagem com o sistema. Cada usuário possui um ou mais papéis.

Papéis

Conjuntos de permissões. Ex: admin, gerente, funcionário.

Permissões

Ações que um papel pode executar. Ex: ler, escrever, excluir.

RBAC simplifica a gestão de acesso. Atribui papéis com permissões aos usuários.

ACL: Action Control List

Sujeito

Usuário ou grupo que tenta acessar o recurso.

Objeto

O recurso protegido, como um arquivo ou API.

Permissões

Ações permitidas: ler, escrever ou executar.

ACL define quem pode acessar um recurso. Determina quais ações podem ser realizadas. Amplamente usada em sistemas de arquivos.

ABAC: Attribute-based Access Control

O que é ABAC?

ABAC usa atributos para conceder permissões. É um modelo dinâmico e flexível. Permissões baseadas em características do usuário, recurso e contexto.

Como Funciona?

- **Sujeito:** Quem acessa o recurso.
- **Recurso:** O objeto protegido.
- **Ação:** O que o sujeito faz.
- **Ambiente:** O contexto do acesso.



DAC: Discretionary Access Control

Proprietário Define Acesso

O dono do recurso decide quem acessa e o que pode ser feito.

Flexibilidade

Usuários ajustam permissões conforme necessário, adaptando o acesso.

Implementação

Comum em sistemas operacionais, como o Windows, para arquivos e pastas.

No DAC, o proprietário tem controle total sobre o recurso. Isso permite conceder ou revogar permissões. Exemplo: Alice concede permissão para Bob editar um documento.

MAC: Mandatory Access Control

Classificação de Segurança

Recursos recebem rótulos de segurança. Ex:
Confidencial, Secreto.

1

2

Níveis de Acesso

Usuários são atribuídos a níveis de segurança. Acesso baseado em comparação.

Políticas Definidas

Sistema controla o acesso. Prioriza a segurança sobre a flexibilidade.

3

Security Best Practices



Segurança e melhores práticas

- Proteção contra ataques
- Bloqueio de tokens
- Boas práticas

Chegando ao fim da nossa jornada!

Exploramos autenticação, autorização e diversos modelos de controle de acesso para sistemas seguros.

Compreender esses conceitos é fundamental para desenvolver aplicações robustas contra ameaças.

Meu muito obrigado pela participação e dedicação durante todo este curso!

